B2B SaaS Platform Testing – Multi-Platform Automation

Candidate: Krishna Tolani
Role: QA Automation Engineering Intern
Company: Bynry Inc.
Case Study: WorkFlow Pro – Multi-Platform Automation

---

1. Introduction

This document describes my approach to testing the WorkFlow Pro B2B SaaS platform.

The platform has several important testing challenges:

- Multiple tenants/companies use the same platform.
- Different users have different roles and permissions.
- The application is available across browsers and mobile devices.
- Backend APIs need to be tested independently.
- UI and API behavior should also be validated together.
- Tests need to be reliable enough to run in CI/CD environments.
- BrowserStack can be used for cross-browser and mobile coverage.

My approach focuses on reliability, maintainability, test isolation, reusable components, and security boundaries between tenants.

---

2. Assumptions

Since some requirements are not fully specified, I have made the following assumptions:

1. A dedicated test/staging environment is available.
2. Test users are available for each tenant and role.
3. Test credentials can be stored securely as environment variables or CI/CD secrets.
4. There is a test-friendly authentication mechanism for users protected by 2FA.

5. The mobile requirement refers to testing the web application on mobile browsers/devices.
6. A project can be created through the provided API.
7. A project can be retrieved through an API or verified through the UI.
8. An appropriate cleanup mechanism exists for deleting test-created projects. The API specification only provides the create endpoint, so the exact cleanup endpoint would need to be confirmed.
9. For tenant isolation, an unauthorized cross-tenant request is expected to return an appropriate error such as 403 or 404.
10. Stable test IDs such as `data-testid` can be added to important UI elements if necessary.

Part 1 – Debugging Flaky Test Code

1.1 Problems Identified

The provided tests can become flaky for several reasons.

1. Immediate URL assertion

The test clicks the login button and immediately checks:

assert page.url == "https://app.workflowpro.com/dashboard"

Login may involve API requests, authentication processing, redirects, and dashboard loading. The assertion may execute before the application has completed these operations.

2. Immediate visibility check

The test uses:

assert page.locator(".welcome-message").is_visible()

The dashboard is dynamically loaded, so the element may not be available yet.

A retrying Playwright assertion is more reliable:

expect(page.locator(".welcome-message")).to_be_visible()

3. Dynamic project loading

The second test uses:

projects = page.locator(".project-card").all()

The project list may still be loading when `.all()` is called. This can result in an incomplete collection or no elements.

The test should first wait for the expected project list state.

4. Hard-coded credentials

The test contains:

"admin@company1.com"
"password123"

Credentials should not be stored directly in source code. They should be provided through environment variables or CI/CD secrets.

5. 2FA is not handled

The requirements state that some users have 2FA. The current test assumes that login always goes directly to the dashboard.

For automation, I would use a dedicated test account with 2FA disabled where appropriate, or a test-only OTP/authentication mechanism.

I would not depend on a human manually entering an OTP in CI/CD.

6. Browser/environment differences

The test only launches Chromium:

browser = p.chromium.launch()

The CI environment may execute tests using different browsers, screen sizes, operating systems, or resource availability.

7. No centralized setup/teardown

Each test creates its own Playwright instance and browser. This makes the code repetitive and harder to maintain.

pytest fixtures can be used to manage browser and authentication setup consistently.

8. Weak selectors

Selectors such as:

#email
#password
.project-card

may be changed when the UI implementation changes.

Where possible, I would prefer accessible locators such as:

page.get_by_label("Email")
page.get_by_role("button", name="Login")

or stable `data-testid` attributes.

1.2 Why These Problems Become More Visible in CI/CD

The local environment may be relatively stable:

Fast computer
Stable network
Same browser
Known screen size
Low test concurrency

CI/CD can be different:

Different CPU/memory
Different browser
Different viewport
Variable network conditions
Multiple tests running simultaneously
Cold browser startup

For example, locally the dashboard might load in 500 ms, while in CI it may take several seconds.

A test that assumes the dashboard is immediately available can therefore pass locally and fail intermittently in CI.

The main solution is to wait for meaningful application states rather than using arbitrary delays such as:

time.sleep(5)

1.3 Corrected Login Test

A simplified corrected version is:

```
import os
import re

from playwright.sync_api import Page, expect


BASE_URL = os.getenv(
    "BASE_URL",
    "https://app.workflowpro.com"
)


def test_user_login(page: Page):

    email = os.environ["COMPANY1_ADMIN_EMAIL"]
    password = os.environ["COMPANY1_ADMIN_PASSWORD"]

    page.goto(
        f"{BASE_URL}/login",
        wait_until="domcontentloaded"
    )

    page.get_by_label("Email").fill(email)
    page.get_by_label("Password").fill(password)

    page.get_by_role(
        "button",
```

```
        name="Login"
    ).click()

    # Wait for the application to complete authentication
    expect(page).to_have_url(
        re.compile(r".*/dashboard/?"),
        timeout=15000
    )

    # Wait for the dynamically loaded dashboard element
    expect(
        page.get_by_test_id("welcome-message")
    ).to_be_visible(timeout=15000)
```

Why this is more reliable

- Credentials are externalized.
- Accessible/stable locators are preferred.
- The test waits for the expected URL.
- The test waits for the dashboard element.
- Arbitrary `sleep()` calls are avoided.
- The timeout is applied to the expected application state.

1.4 Corrected Multi-Tenant Test

```
import os
import re

from playwright.sync_api import Page, expect


BASE_URL = os.getenv(
    "BASE_URL",
    "https://app.workflowpro.com"
)


def test_multi_tenant_access(page: Page):

    email = os.environ["COMPANY2_USER_EMAIL"]
    password = os.environ["COMPANY2_USER_PASSWORD"]
```

```
    page.goto(
        f"{BASE_URL}/login",
        wait_until="domcontentloaded"
    )

    page.get_by_label("Email").fill(email)
    page.get_by_label("Password").fill(password)

    page.get_by_role(
        "button",
        name="Login"
    ).click()

    expect(page).to_have_url(
        re.compile(r".*/dashboard/?"),
        timeout=15000
    )

    projects = page.get_by_test_id("project-card")

    # Wait for dynamically loaded project data.
    expect(projects.first).to_be_visible(
        timeout=15000
    )

    count = projects.count()

    assert count > 0, "Expected at least one project"

    for index in range(count):

        project = projects.nth(index)

        expect(project).to_contain_text(
            "Company2"
        )
```

Important security consideration

Checking that every visible project contains `"Company2"` is useful as a UI-level check, but it is not sufficient for security validation.

Tenant isolation should also be tested at the API/backend level.

For example:

Company 1 project → ID 123

Company 2 attempts:
GET /api/v1/projects/123

Expected:
403 Forbidden OR 404 Not Found

The exact expected response should be confirmed with the backend/API specification.

Part 2 – Test Automation Framework Design

2.1 Proposed Framework Structure

I would use Python, pytest, and Playwright with a structure such as:

```
workflowpro-automation/
│
├── tests/
│   ├── ui/
│   │   ├── test_login.py
│   │   ├── test_projects.py
│   │   └── test_permissions.py
│   │
│   ├── api/
│   │   └── test_projects_api.py
│   │
│   └── integration/
│       └── test_project_creation.py
│
├── pages/
│   ├── login_page.py
│   ├── dashboard_page.py
```

```
│   └── project_page.py
│
├── api/
│   ├── client.py
│   └── projects.py
│
├── fixtures/
│   ├── auth.py
│   ├── tenants.py
│   └── projects.py
│
├── test_data/
│   ├── users.json
│   └── projects.json
│
├── config/
│   └── environments.yaml
│
├── reports/
│
├── .github/
│   └── workflows/
│       └── tests.yml
│
├── conftest.py
├── pytest.ini
├── requirements.txt
├── .gitignore
└── README.md
```

2.2 Responsibilities of Each Folder

`tests/`

Contains actual test scenarios.

Examples:

test_login.py
test_projects.py
test_permissions.py
test_projects_api.py

Tests should primarily describe business behavior.

---

pages/

Contains Page Object classes.

For example:

```
class LoginPage:

    def __init__(self, page):
        self.page = page

    def login(self, email, password):

        self.page.get_by_label("Email").fill(email)
        self.page.get_by_label("Password").fill(password)

        self.page.get_by_role(
            "button",
            name="Login"
        ).click()
```

This keeps UI implementation details separate from test logic.

---

api/

Contains reusable API clients.

For example:

```
class ProjectsAPI:

    def create_project(
        self,
        token,
        tenant_id,
        project_data
    ):
```

```
    # Send POST request
    pass
```

`fixtures/`

Contains reusable setup and test resources.

Examples:

- authenticated users
- tenant configuration
- project setup
- cleanup

pytest fixtures can help avoid repeating setup code in every test.

`test_data/`

Contains non-sensitive test data.

Passwords and API tokens should **not** be stored here. Sensitive information should come from environment variables or CI/CD secret storage.

`config/`

Contains environment and execution configuration.

For example:

```
environments:

  staging:
    web_url: "https://app.workflowpro.com"
    api_url: "https://api.workflowpro.com"

tenants:

  company1:
    tenant_id: "company1"
```

```
  company2:
    tenant_id: "company2"
```

2.3 Handling Different Roles

The application has:

Admin
Manager
Employee

I would model users according to their tenant and role.

Example:

```
USERS = {
    "company1_admin": {
        "tenant": "company1",
        "role": "admin"
    },

    "company1_manager": {
        "tenant": "company1",
        "role": "manager"
    },

    "company1_employee": {
        "tenant": "company1",
        "role": "employee"
    }
}
```

Tests can then validate permissions.

For example:

Admin:
Create project → Allowed

Manager:

Create project → Depends on requirement

Employee:
Create project → Not allowed

The exact permissions should be confirmed with the product requirements.

2.4 Configuration Management

I would separate configuration from test code.

Examples of environment variables:

BASE_URL
API_URL

COMPANY1_ADMIN_EMAIL
COMPANY1_ADMIN_PASSWORD

COMPANY2_USER_EMAIL
COMPANY2_USER_PASSWORD

COMPANY1_API_TOKEN
COMPANY2_API_TOKEN

BROWSERSTACK_USERNAME
BROWSERSTACK_ACCESS_KEY

Secrets should be stored in CI/CD secret management and never committed to GitHub.

2.5 Browser Strategy

The framework should support:

Web:
- Chromium

- Firefox
- WebKit/Safari equivalent

Mobile:
- Android
- iOS

For local/CI browser coverage, Playwright can run the main web browser suite.

For real-device/browser coverage, BrowserStack can be integrated.

I would use a risk-based execution strategy rather than running every device for every code change.

For example:

Pull Request:
Critical API + Chromium smoke tests

Main branch:
Chromium + Firefox + WebKit

Nightly:
Full browser + mobile BrowserStack regression

This balances coverage, execution time, and BrowserStack cost.

2.6 CI/CD Strategy

A simplified pipeline would be:

Developer pushes code
↓
CI pipeline starts
↓
Install dependencies
↓
Run API smoke tests
↓
Run UI smoke tests

↓
Run browser tests
↓
Run BrowserStack tests when required
↓
Generate test report
↓
PASS / FAIL

For failed tests, I would collect useful debugging information such as:

- screenshots
- Playwright traces
- logs
- test reports

This makes failures easier to investigate.

2.7 Missing Requirements / Questions

Because the requirements are incomplete, I would clarify the following before implementing the full framework.

Authentication

1. How is 2FA handled in test environments?
2. Are dedicated automation accounts available?
3. Can we create authenticated sessions without going through the UI every time?

Test Data

1. How are test users created?
2. How are projects created and deleted?
3. Can tests create their own data through APIs?
4. Can multiple tests use the same tenant simultaneously?
5. How should test data be cleaned up?

Tenant Isolation

1. How is tenant identity represented?

2. Is tenant information stored in the token, URL, or request header?
3. What should happen when a user requests another tenant's resource?
4. Can a user belong to multiple tenants?

Browser/Mobile

1. Which browser versions are officially supported?
2. Which iOS and Android devices are required?
3. Is mobile testing for responsive web or native applications?

CI/CD

1. Which CI/CD platform is used?
2. How many tests can run in parallel?
3. What is the maximum acceptable pipeline duration?
4. Which tests should run on every pull request?

Reporting

1. Is HTML, JUnit, or Allure reporting required?
2. Where should reports be stored?
3. Should screenshots and videos be stored for failed tests?

Part 3 – API + UI Integration Test

3.1 Testing Strategy

The complete flow will use three layers:

API
↓
Create test data quickly

Web UI
↓
Verify user-facing behavior

Mobile
↓
Verify mobile accessibility

Security/API
↓
Verify tenant isolation

The API is used for efficient test-data setup, while the UI validates what the user actually sees.

3.2 Test Flow

The test will perform:

1. Create project for Company 1 through API
↓
2. Capture project ID
↓
3. Login as Company 1
↓
4. Verify project in web UI
↓
5. Verify project on mobile
↓
6. Login/use credentials for Company 2
↓
7. Verify Company 2 cannot see project
↓
8. Verify Company 2 cannot access project through API

3.3 API Project Creation

A reusable helper can create the project:

```
import uuid


def create_project(api, token, tenant_id):

    project_name = (
```

```
        f"Automation Project "
        f"{uuid.uuid4().hex[:8]}"
    )

    response = api.post(
        "/api/v1/projects",
        headers={
            "Authorization": f"Bearer {token}",
            "X-Tenant-ID": tenant_id
        },
        data={
            "name": project_name,
            "description": "Created by automation",
            "team_members": []
        }
    )

    assert response.ok

    body = response.json()

    assert body["name"] == project_name
    assert body["status"] == "active"

    return body
```

A unique project name helps prevent conflicts when tests run in parallel.

3.4 Integration Test Structure

```
def test_project_creation_flow():

    # 1. API
    # Create a project for Company 1.
    project = create_project(
        api=api,
        token=company1_token,
        tenant_id="company1"
    )
```

```
    project_id = project["id"]
    project_name = project["name"]


    # 2. WEB UI
    # Login as Company 1.
    # Open the projects page.
    # Verify the created project is visible.


    # 3. MOBILE
    # Run the same important validation
    # on a mobile BrowserStack session.


    # 4. SECURITY
    # Authenticate as Company 2.
    # Verify Company 1's project is not visible.
    # Also verify API access is denied.
```

3.5 Example Web UI Validation

```
def verify_project_in_ui(page, project_id, project_name):

    project_card = page.get_by_test_id(
        f"project-{project_id}"
    )

    expect(project_card).to_be_visible(
        timeout=30000
    )

    expect(project_card).to_contain_text(
        project_name
    )
```

The long timeout is used here only as an example for an asynchronously loaded project. I would tune timeout values based on actual application performance rather than using arbitrary large values everywhere.

3.6 Tenant Isolation Test

The most important security scenario is:

Company 1 creates project 123
↓
Company 2 attempts to access project 123
↓
Access must be denied

API-level validation:

```
response = api.get(
    f"/api/v1/projects/{project_id}",
    headers={
        "Authorization": f"Bearer {company2_token}",
        "X-Tenant-ID": "company2"
    }
)

assert response.status in [403, 404]
```

The exact expected status code should be confirmed with the backend/API contract.

UI-level validation:

```
project = page.get_by_test_id(
    f"project-{project_id}"
)

expect(project).not_to_be_visible()
```

This provides both:

- backend security validation
- frontend visibility validation

3.7 Mobile / BrowserStack Approach

For mobile validation, I would execute the same important business scenario against a BrowserStack mobile browser/device.

Conceptually:

Test
↓
BrowserStack
↓
Android / iOS device
↓
Open WorkFlow Pro
↓
Login
↓
Open Projects
↓
Verify project

The exact BrowserStack device/browser configuration would be maintained separately from business test logic.

This allows the same test scenario to be executed against multiple platforms without duplicating the complete test.

3.8 Test Data Management

Test data should be isolated between tests.

For example:

```
project_name = (
    f"Automation Project "
    f"{uuid.uuid4().hex[:8]}"
)
```

This prevents two parallel tests from creating projects with exactly the same name.

The preferred lifecycle is:

If the application provides an API for deleting projects, I would use it during teardown.

If such an API does not exist, I would ask the team what approved cleanup mechanism should be used.

3.9 Edge Cases

I would also consider the following scenarios.

API failures

- 400 – invalid request
- 401 – invalid/expired authentication
- 403 – unauthorized tenant access
- 404 – project not found
- 429 – rate limiting
- 500 – server error
- network timeout

UI issues

- Slow project loading
- Empty project list
- Project loading after a delay
- Browser refresh

- Expired session
- Different viewport sizes
- Responsive layout issues

Tenant security

Company 1 token + Company 1 tenant
→ Allowed

Company 2 token + Company 2 tenant
→ Allowed

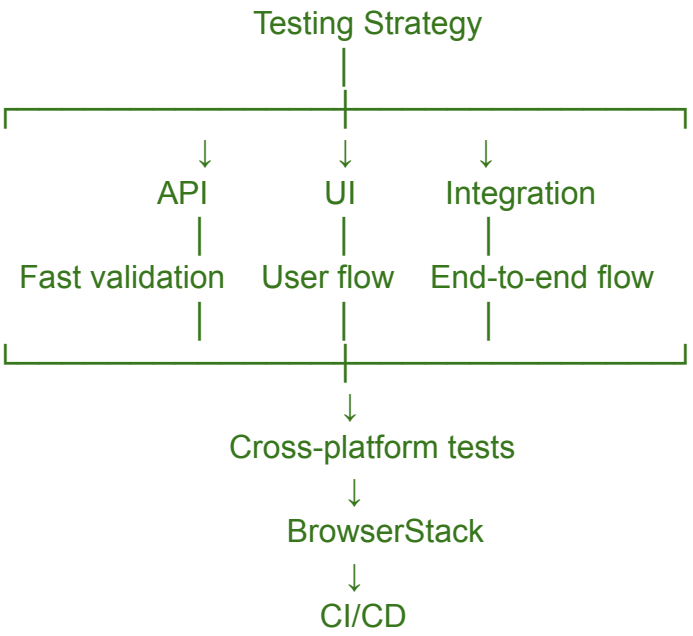
Company 1 token + Company 2 tenant
→ Denied

Company 2 token + Company 1 project
→ Denied

---

4. Overall Testing Strategy

My overall approach would be based on different levels of testing.



API tests would provide fast feedback.

UI tests would validate important user workflows.

Integration tests would validate that different layers work together.

BrowserStack would provide additional browser and mobile coverage.

---

5. Reliability and Flaky Test Prevention

To reduce flaky tests, I would follow these principles:

1. Use Playwright's built-in waiting and retrying assertions.
2. Avoid unnecessary `sleep()` calls.
3. Use stable locators.
4. Wait for meaningful application states.
5. Generate isolated test data.
6. Avoid sharing mutable test data between parallel tests.
7. Handle authentication consistently.
8. Capture traces/screenshots when failures occur.
9. Keep tests independent.
10. Investigate the root cause of flaky tests rather than simply increasing retries.

Retries can sometimes help with temporary infrastructure problems, but they should not hide genuine application failures.

---

6. Why API + UI Testing?

I would use both because they answer different questions.

API test

Answers:

> "Did the backend correctly create the project?"

UI test

Answers:

> "Can the user actually see and interact with the project?"

Integration test

Answers:

> "Does the complete flow from backend to frontend work correctly?"

This combination gives better coverage than relying only on UI tests.

---

7. Final Summary

The main priorities for this application are:

Reliability

Tests should not depend on timing assumptions.

Maintainability

Page Objects, API clients, fixtures, and reusable utilities should reduce duplication.

Security

Tenant isolation should be tested at both UI and API levels.

Cross-platform coverage

Important workflows should be validated across supported browsers and mobile platforms.

Test data isolation

Tests should create unique data and clean it up to support parallel execution.

CI/CD

A smaller smoke suite can run on pull requests while broader cross-browser/mobile regression tests can run on scheduled builds or release pipelines.

My overall goal would be to build a framework that provides fast feedback to developers while maintaining sufficient coverage for critical business and security scenarios.

---

8. Limitations and Next Steps

The provided case study does not define some important API and infrastructure details, particularly:

- Authentication/2FA implementation
- Project retrieval endpoint
- Project deletion endpoint
- Exact browser/device matrix
- Expected HTTP status codes for cross-tenant access
- Test environment setup
- CI/CD platform
- BrowserStack session limits

Before implementing this framework in a real project, I would clarify these requirements with the development/product/DevOps teams.

The implementation would then be adjusted to the actual API contracts and application behavior.